LAPORAN PRAKTIKUM ALGORITMA

DAN PEMROGRAMAN 2

MODUL 10

Pencarian Nilai Max/Min

Disusun oleh:

Ahmad Malik Arrayyan

109082500116

S1IF-13-02

Asisten Praktikum

Muhammad Agha Zulfadhi

Renisa Assyifa Putri

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

Tugas

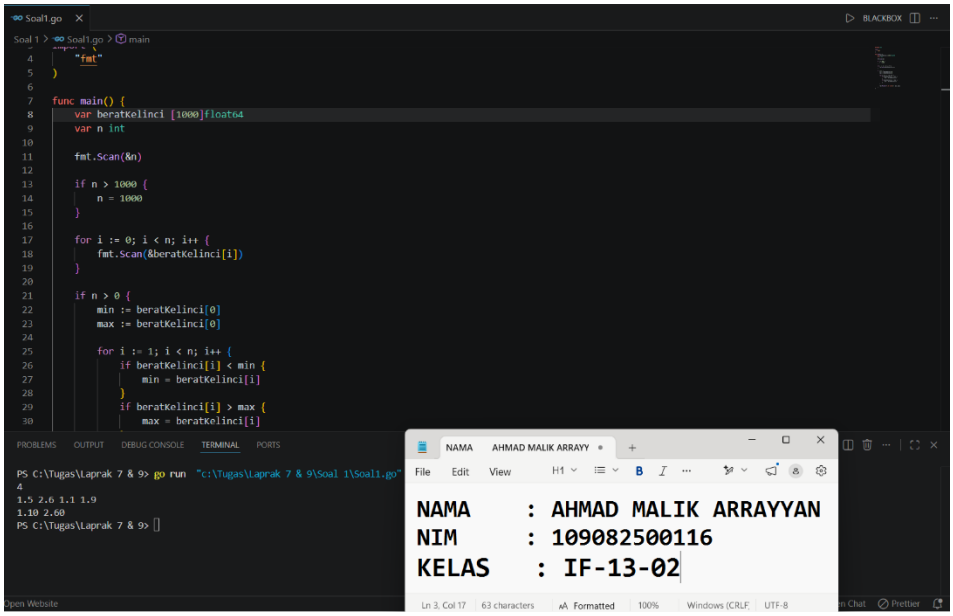
1. **Tugas 1**

   Source Code

```
package main

import (
    "fmt"
)

func main() {
    var beratKelinci [1000]float64
    var n int

    fmt.Scan(&n)

    if n > 1000 {
        n = 1000
    }

    for i := 0; i < n; i++ {
        fmt.Scan(&beratKelinci[i])
    }

    if n > 0 {
        min := beratKelinci[0]
        max := beratKelinci[0]

        for i := 1; i < n; i++ {
            if beratKelinci[i] < min {
                min = beratKelinci[i]
            }
            if beratKelinci[i] > max {
                max = beratKelinci[i]
            }
```

```
            }


            fmt.Printf("%.2f %.2f\n", min, max)
        }
    }
```

Screenshoot program



Deskripsi program

Program ini dirancang untuk menemukan nilai tumpuan atau nilai ekstrem—yaitu berat terkecil dan berat terbesar—dari sekumpulan data kelinci yang diinputkan. Penjelasan kodenya dimulai dengan penyiapan wadah berupa **Array Statis** berkapasitas 1000 data. Program ini memiliki sistem keamanan di mana jika pengguna memasukkan jumlah data lebih dari kapasitas, sistem secara otomatis akan membatasinya di angka 1000 agar tidak terjadi kebocoran memori atau *error*.

Logika inti dari pencariannya sangat efisien. Alih-alih melakukan pengurutan (*sorting*) yang memakan waktu lama, program ini menggunakan metode **Linear Scan**. Pertama, program mengasumsikan bahwa kelinci pertama adalah yang terkecil sekaligus yang terbesar sebagai titik acuan. Kemudian, melalui perulangan, program akan membandingkan data acuan tersebut dengan sisa kelinci lainnya satu per satu. Jika ditemukan kelinci yang lebih ringan, variabel min akan diperbarui; sebaliknya, jika ada yang lebih berat, variabel max yang diperbarui. Hasil akhirnya ditampilkan dengan presisi dua angka di belakang koma, memberikan informasi yang cepat dan akurat mengenai rentang berat badan dalam populasi kelinci tersebut.

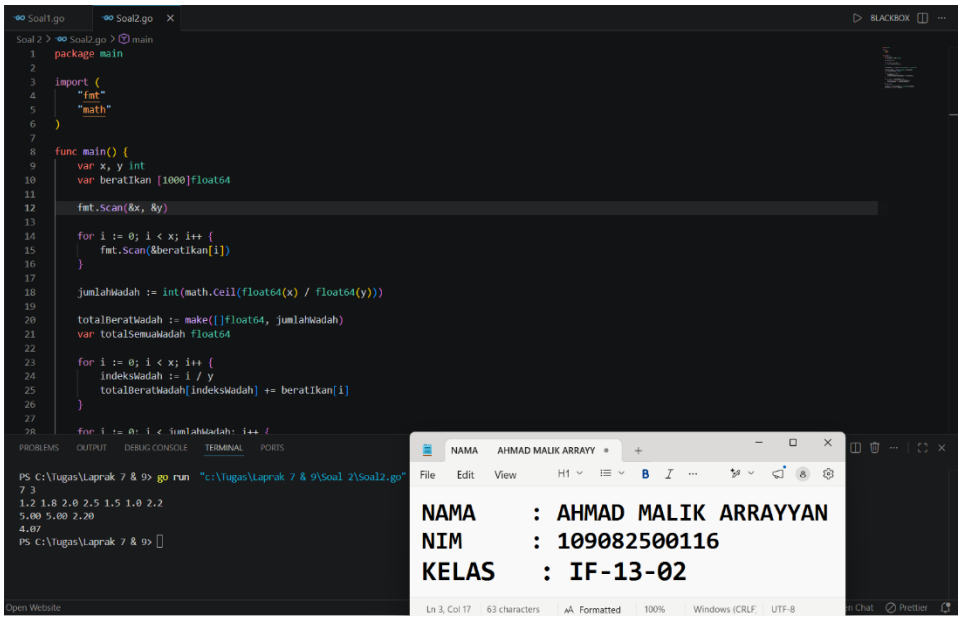
2. **Tugas 2**

Source Code

```
package main

import (
    "fmt"
    "math"
)

func main() {
    var x, y int
    var beratIkan [1000]float64

    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Scan(&beratIkan[i])
    }

    jumlahWadah := int(math.Ceil(float64(x) /
float64(y)))

    totalBeratWadah := make([]float64, jumlahWadah)
    var totalSemuaWadah float64

    for i := 0; i < x; i++ {
        indeksWadah := i / y
        totalBeratWadah[indeksWadah] += beratIkan[i]
    }

    for i := 0; i < jumlahWadah; i++ {
        fmt.Printf("%.2f ", totalBeratWadah[i])
        totalSemuaWadah += totalBeratWadah[i]
```

```
        }
        fmt.Println()


        rataRata := totalSemuaWadah / float64(jumlahWadah)
        fmt.Printf("%.2f\n", rataRata)
    }
```

Screenshoot program



Deskripsi program

Program ini dirancang untuk mensimulasikan proses pembagian hasil panen ikan ke dalam beberapa wadah dengan kapasitas yang ditentukan. Pertama, program menerima dua input: jumlah seluruh ikan (**x**) dan kapasitas maksimum setiap wadah (**y**). Data berat setiap ikan disimpan di dalam sebuah array statis. Untuk menentukan berapa banyak wadah yang dibutuhkan, saya menggunakan fungsi math.Ceil. Fungsi ini sangat penting karena ia melakukan pembulatan ke atas; misalnya, jika ada 11 ikan dan setiap wadah hanya muat 5, maka secara logika kita butuh 3 wadah agar semua ikan tertampung.Bagian paling cerdas dari kode ini adalah cara distribusinya menggunakan operasi pembagian bilangan bulat (i / y). Dengan rumus ini, komputer secara otomatis mengelompokkan ikan ke indeks wadah yang tepat; misalnya, ikan ke-0 sampai ke-4 akan masuk ke wadah indeks 0, lalu ikan ke-5 akan masuk ke wadah indeks 1. Setelah semua ikan terbagi, program menghitung total berat di setiap wadah secara spesifik, lalu di bagian akhir, program menghitung rata-rata berat per wadah dengan menjumlahkan seluruh isi wadah dan membaginya dengan jumlah wadah yang digunakan. Pendekatan ini memastikan distribusi data yang rapi dan perhitungan statistik yang akurat untuk setiap kelompok data.

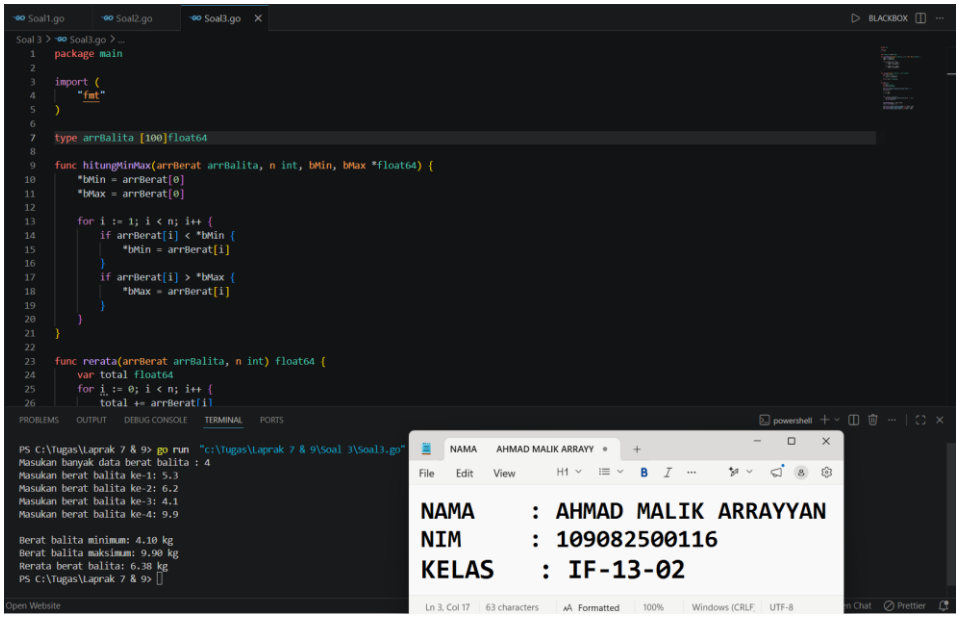
3. **Tugas 3**

Source Code

```
package main

import (
    "fmt"
)

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}
```

```
func main() {
    var n int
    var data arrBalita
    var min, max float64

    fmt.Print("Masukan banyak data berat balita : ")
    fmt.Scan(&n)

    if n > 100 {
        n = 100
    }

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&data[i])
    }

    hitungMinMax(data, n, &min, &max)
    rata := rerata(data, n)

    fmt.Printf("\nBerat balita minimum: %.2f kg\n",
min)
    fmt.Printf("Berat balita maksimum: %.2f kg\n", max)
    fmt.Printf("Rerata berat balita: %.2f kg\n", rata)
}
```

Screenshoot program



Deskripsi program

Program ini dirancang untuk mengolah data statistik kesehatan balita secara otomatis dengan fokus pada tiga informasi utama: berat terkecil, berat terbesar, dan rata-rata. Inti dari program ini adalah penggunaan **Array Statis** dengan tipe data kustom arrBalita untuk menyimpan hingga 100 data berat badan. Keunggulan teknis kode ini terletak pada fungsi hitungMinMax yang menggunakan **Pointer**. Alih-alih membuat dua fungsi terpisah, saya menggunakan pointer agar satu fungsi tersebut bisa langsung memperbarui nilai variabel min dan max di memori utama secara bersamaan.

Proses pencarian nilai ekstrem dilakukan dengan membandingkan setiap data di dalam array secara berurutan; jika ditemukan angka yang lebih kecil dari simpanan sementara, maka nilai minimum diperbarui, begitu pula sebaliknya untuk nilai maksimum. Selain itu, terdapat fungsi rerata yang menjumlahkan seluruh data berat badan dan membaginya dengan jumlah balita untuk mendapatkan nilai tengah yang akurat. Dengan struktur yang modular dan penggunaan pointer ini, program menjadi sangat efisien dalam memproses data medis sederhana karena mampu menghasilkan banyak output statistik hanya dengan satu kali pemindaian data.